

TOPIC 3

ERD, UML
CLASS
DIAGRAM &
BEHAVIORAL
UML
DIAGRAMS

Learning Outcomes

By the end of Week 04, students should be able to:

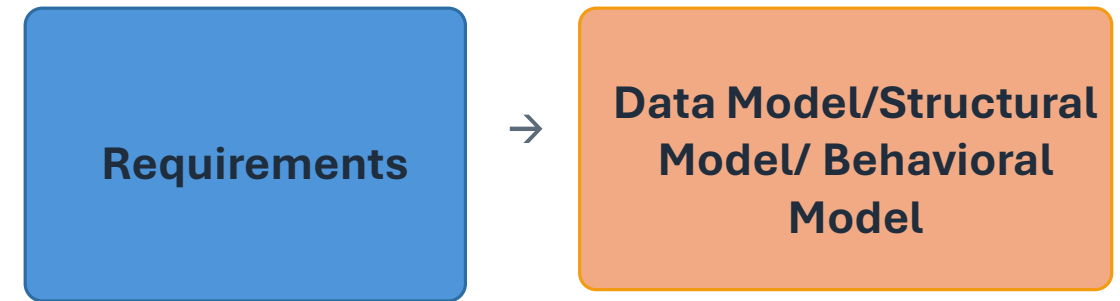
1. Explain the purpose of ERD and Class Diagram
2. Distinguish between **data modeling** and **object-oriented modeling**
3. Identify **entities, attributes, keys, relationships, classes, methods, and diagram notations**
4. Draw simple diagrams from a short scenario
5. Apply cardinality/optionality in ERD and multiplicity/visibility in UML Class Diagram

Remember box

- **ERD asks:** What data must the system store?
- **Class Diagram asks:** What software objects exist and what can they do?
- **UML asks:** How can we visualize system structure and behavior?

Why Analysts Use Diagrams

- Diagrams convert long verbal descriptions into visual models.
- They help analysts communicate structure, boundaries, relationships, and logic more clearly.
- A good diagram supports shared understanding between analysts, developers, and stakeholders.
- Diagrams do not replace thinking; they make thinking visible and discussable.



Note

In analysis, a model is a **simplified representation of reality** created for understanding and decision-making.

ERD = Data, Class Diagram = Structural, Use Case/Activity/Sequence/State = Behavioral.

SESSION 1

ERD & CLASS DIAGRAM

Where ERD and Class Diagram Fit

- During analysis, we study **what the system must support and what information it must manage**.
- ERD is mainly used for **conceptual data modeling**.
- Class Diagram is widely used in **object-oriented analysis and design**.
- Both are useful, but they answer **different questions**.

Memory cue

- **ERD → database thinking**
- **Class Diagram → software thinking**

Data View vs Object View

Data view

- Focuses on data items, data structure, and business relationships
- Asks what must be stored and linked

Object view

- Object view focuses on what objects know and what operations they can perform

Key insight

- A real-world system can be modeled from multiple perspectives.
- No single diagram explains everything.

What is an Entity Relationship Diagram (ERD)?

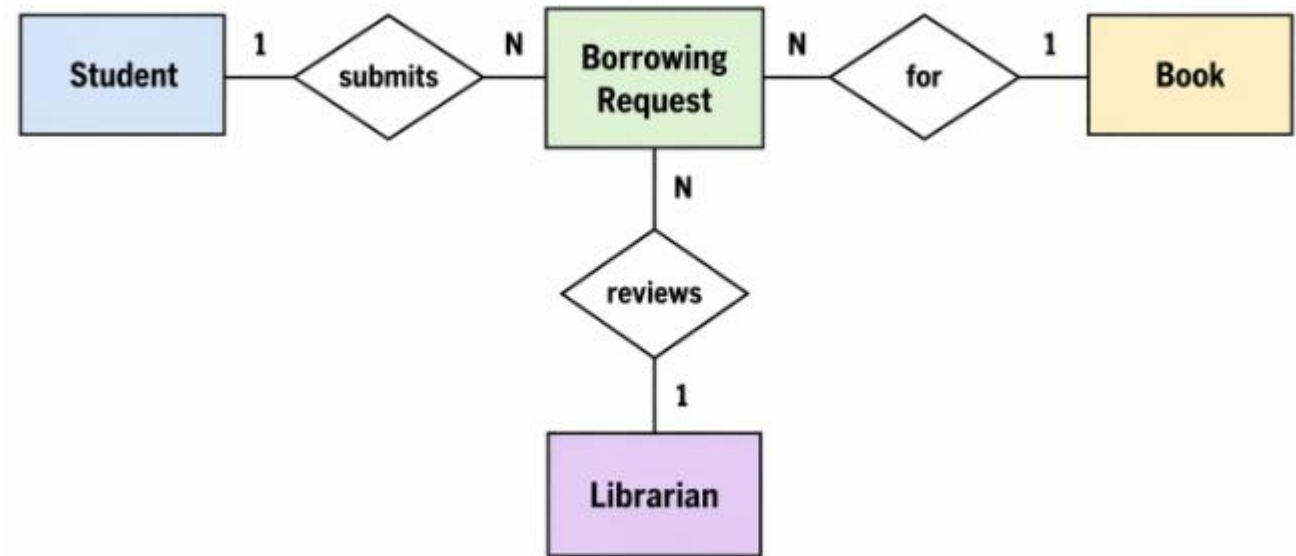
- An Entity-Relationship Diagram (ERD) is a conceptual model used to represent the **data structure** of a system: **entities, attributes, and relationships**.
- It helps determine what data the system needs to store, how the data is linked together, and is a crucial step before database design.
- ERD is particularly useful when clarifying business rules before moving to relational tables or before moving to a relational database schema.

Components of an ERD

- **Entity:** a thing the system stores data about
- **Attribute:** a property of an entity
- **Relationship:** a connection between entities
- **Cardinality:** how many instances can participate
- **Participation / Optionality:** whether participation is mandatory or optional

Note:

Entity rectangle; Attribute oval, Key attribute underlined, Relationship diamond, Cardinality labels



Entity in ERD

- An **entity** may be a person, object, event, place, or concept.

Examples: Student, Lecturer, Company, Internship Request, Invoice, Order.

- A **strong entity** has its own key.
- A **weak entity** depends on another entity for identification.

Academic note

- Not every noun in a scenario becomes an entity.
- A good entity usually represents a **stable business concept** with meaningful data.

Remember

- “Important noun + stored data + repeated occurrence” often suggests an entity.

Attributes in ERD

Attributes describe an entity's properties.

Example:

- Student(StudentID, FullName, PhoneNumbers, Email, DateOfBirth)
- Composite attribute = FullName including FirstName, LastName; multivalued = PhoneNumbers; derived = Age from DateOfBirth
- Attribute types may include:
 - **Key attribute**
 - **Composite attribute**
 - **Multivalued attribute**
 - **Derived attribute**

Academic note

- Attributes should be meaningful, non-redundant, and relevant to the business need.
- Avoid storing the same fact in multiple places without reason.

Keys and Identifiers

- A **key** uniquely identifies an entity instance.
- A **primary key** is the main identifier.
- Keys are essential because relationships are often built through identifiers.
- Good keys support accuracy, consistency, and later table design.

Example

- StudentID identifies one student
- RequestID identifies one internship request

Remember

- If two records can still be confused, the key is not good enough.
- In conceptual ERD, keys identify entities. In logical ERD, foreign keys are added to implement relationships.

Relationships

- A relationship shows how entities are connected in the business domain.
- Examples:
 - Student submits InternshipRequest
 - Lecturer reviews InternshipRequest
 - Unary: Employee supervises Employee; Binary: Student submits Request; Ternary: Supplier supplies Part to Project.
- Relationships can be **unary**, **binary**, or **ternary** depending on how many entity sets participate.

Academic note

- A relationship should express a **business rule**, not just a visual line.

Cardinality and Optionality

Cardinality answers: how many?

- 1:1
- 1:M
- M:N

Participation / Optionality answers: must or may?

- Total participation
- Partial participation

Example

- One Company may host many Students.
- A Passport belongs to one Person.
- A Student can enroll in many Courses; one Course can have many Students.

Remember

- Cardinality = quantity
- Optionality = obligation

1:1

Passport - Person

1:N

Company - Student

M:N

Student - Course

Many-to-Many and Associative Entity

- A many-to-many relationship often cannot be implemented directly in a relational database.
- It is usually resolved by introducing an **associative entity**.

Example:

- Student $\leftrightarrow$ Course becomes Student — Enrollment — Course
- The associative entity may carry important data such as date, status, score, or approval result.

Conceptual ERD may show M:N directly. Logical ERD resolves M:N using associative entity.

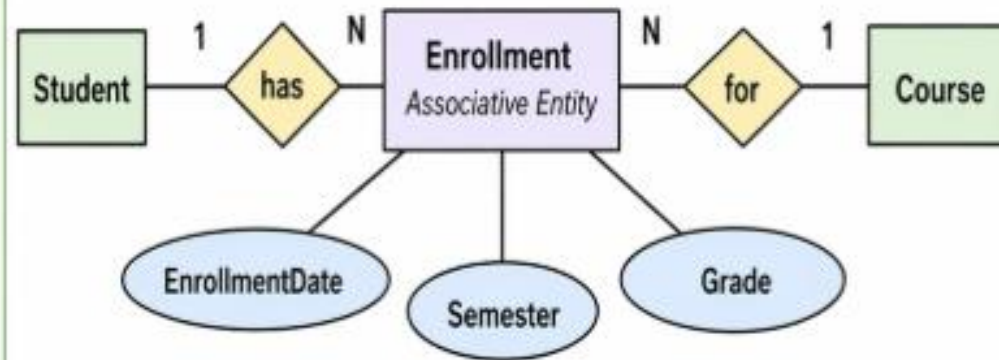
Many-to-Many and Associative Entity

A. Many-to-Many Relationship (M:N)



One student can enroll in many courses, and one course can have many students.

B. Resolving M:N with an Associative Entity



Enrollment is an associative entity used to resolve the many-to-many relationship.

Enrollment is created because the relationship has its own data such as semester, grade, enrollment date.

Steps to Draw an ERD

1. Read the scenario and highlight important nouns and business events
2. Identify candidate entities
3. Identify attributes and keys
4. Identify relationships and business rules
5. Add cardinality and participation
6. Check redundancy and clarity
7. Validate the ERD by reading each relationship as a business rule.

ERD Naming and Quality Rules

- Use **clear entity names** and **meaningful attribute names**
- Keep naming consistent
- Avoid duplicate entities that represent the same concept
- Show relationships based on real business meaning
- Do not overload one diagram with too much detail

Common mistakes

- Using unclear names such as “Data1”, “Info”, “Thing”
- Forgetting keys
- Missing cardinality
- Turning every noun into an entity
- Missing relationship names
- Using attributes as entities
- Not resolving M:N in logical ERD
- Inconsistent singular/plural naming.

ERD

For the Internship Scenario

Scenario :

A university wants to manage internship registration:

- A student may submit many internship requests over time, but each request is submitted to one company.
- Each request has a request date, start date, end date, and status.
- A lecturer reviews the request.
- A coordinator gives final approval.
- A request may include many uploaded documents.
- A company may host many students over time

ERD

For the Internship Scenario

Possible entities	Possible relationships	Business rule examples
Student (StudentID, FullName, Email, Phone, ProgramMajor)	Student submits InternshipRequest (1:M)	A student may submit many internship requests over time, but each internship request belongs to one student.
Company (CompanyID, CompanyName, Address, ContactPerson, ContactEmail, ContactPhone)	Company receives InternshipRequest (1:M)	A company may receive many internship requests, but each internship request is submitted to one company.
InternshipRequest (RequestID, RequestDate, StartDate, EndDate, Status, Remarks)	InternshipRequest is reviewed by Lecturer (M:1)	One lecturer can review many internship requests, but each internship request is reviewed by one lecturer.

ERD

For the Internship Scenario

Possible entities	Possible relationships	Business rule examples
Lecturer (LecturerID, FullName, Email, Phone, Department)	Lecturer reviews InternshipRequest (1:M)	A lecturer may review many internship requests, but each request is reviewed by one lecturer.
Coordinator (CoordinatorID, FullName, Email, Phone)	Coordinator gives final approval for InternshipRequest (1:M)	A coordinator may approve many internship requests, but each request receives final approval from one coordinator.
Document (DocumentID, DocumentName, DocumentType, UploadDate, FilePath)	InternshipRequest includes Document (1:M)	One internship request may include many uploaded documents, but each document belongs to one internship request.

ERD

For the Internship Scenario

The relationship between **Student** and **Company** is conceptually **many-to-many**:

- One student may apply for many internship positions
- One internship position may receive applications from many students

This many-to-many relationship is resolved by the associative entity **InternshipRequest**.

So:

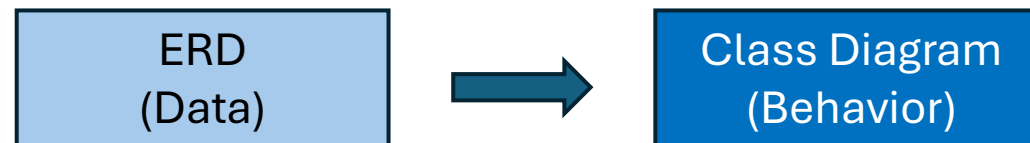
- **Student 1 : M InternshipRequest**
- **Company 1 : M InternshipRequest**

Transition to Class Diagram

- ERD helps us understand the **data world**.
- Class Diagram helps us understand the **software world**.
- Some concepts appear in both, but Class Diagram adds **behavior** through methods.
- Therefore, a Class Diagram is not just an ERD with boxes.

Memory cue

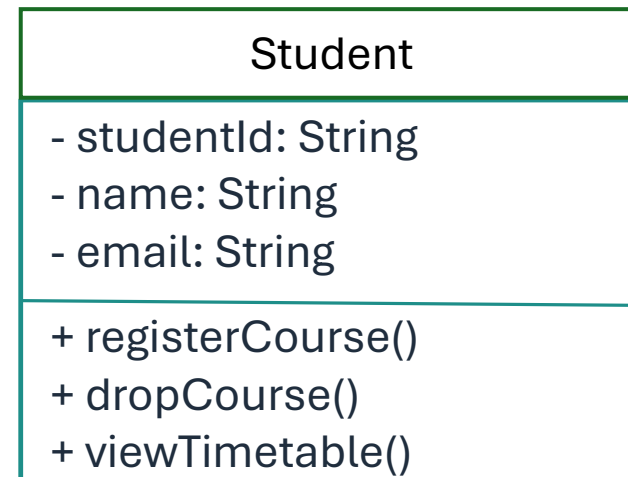
- ERD = what information exists
- Class Diagram = what software objects know and do
- Entities and classes may look similar, but they are not always mapped one-to-one.



What Is a Class Diagram?

- A Class Diagram shows the **static structure** of an object-oriented system.
- A Class Diagram does not show the sequence of actions; it shows structural relationships among classes.
- It includes **classes, attributes, methods, and relationships**.
- It supports object-oriented thinking and helps bridge analysis and design.
- At the analysis phase, a class diagram helps transition from business requirements to object-oriented thinking before moving on to detailed design.

Class = data + behavior



Structure of a Class

A class usually contains:

- **Class name**
- **Attributes:** data held by the object
- **Methods / Operations:** actions the object can perform

Example

Student

- studentId
- fullName
- email
- submitRequest()
- viewStatus()

Academic note

- A class is a **blueprint**; an object is an instance created from that blueprint.

Relationships in Class Diagrams

- **Association:** a general link between classes
- **Aggregation:** a weak whole-part relationship
- **Composition:** part cannot exist independently from whole
- **Inheritance / Generalization:** one class specializes another

Example

- User → Student, Lecturer, Coordinator
- Lecturer contains Department
- Student submits InternshipRequest

Association

Example:

Student — InternshipRequest

Aggregation

Example:

Department ◇— Lecturer

Composition

Example:

Order ◆— OrderLine

Inheritance

Example:

Student —|> User

Multiplicity in UML Class Diagram

Definition

- Multiplicity indicates how many objects of one class may be associated with an object of another class.
- Multiplicity is written near the opposite class end and tells how many objects of that class can be linked

Common multiplicities

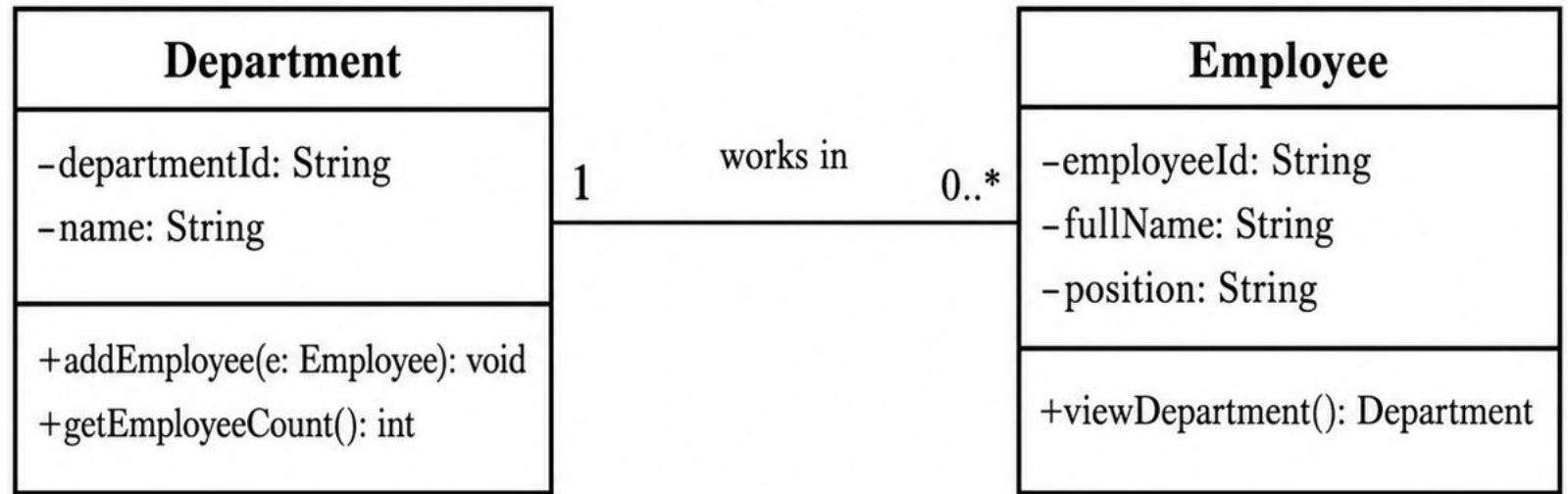
- **1** = exactly one
- **0..1** = zero or one
- **1..*** = one or many
- **0..*** = zero or many

Why it matters

- It expresses business rules clearly.
- It helps analysts and designers understand the structure of relationships.
- It prevents ambiguity in system modeling.

Multiplicity in UML Class Diagram

Example Department — Employee



Interpretation: One Department may have 0..* Employees.
Each Employee belongs to 1 Department.

Visibility in UML Class Diagram

Definition

Visibility shows the access level of attributes and operations in a class.

Common visibility symbols

+ public = accessible from any class

- private = accessible only within the same class

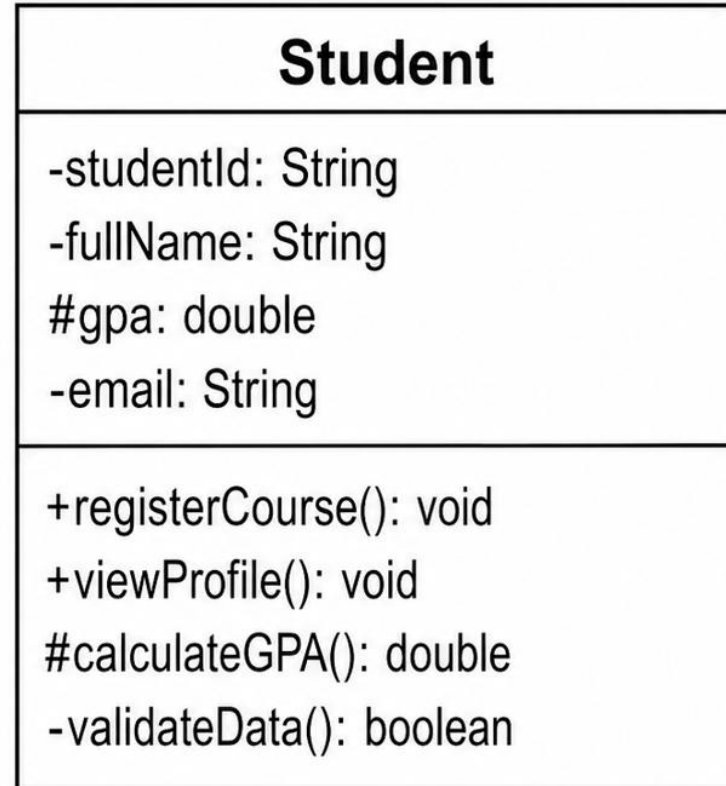
protected = accessible within the same class and its subclasses

Why it matters

- It supports encapsulation.
- It controls access to data and behavior.
- It improves object-oriented design quality.

Visibility in UML Class Diagram

Example



Visibility Symbols

+ public = accessible from any class

- private = accessible only within the same class

protected = accessible within the class and its subclasses

This example shows how visibility controls access to attributes and operations.

Multiplicity and Visibility

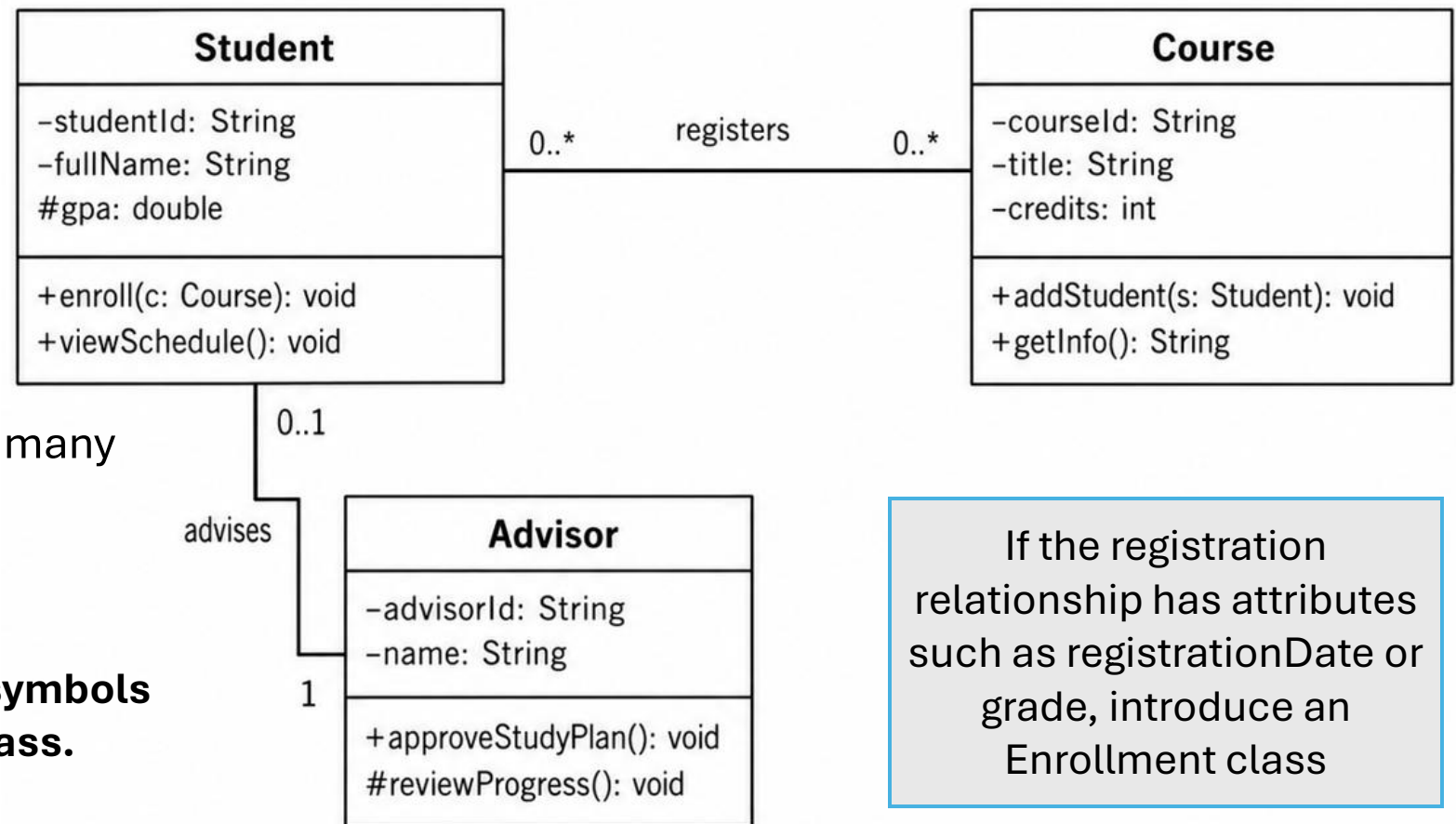
This example shows:

- **Multiplicity** on the relationship between classes
- **Visibility** inside each class

Interpretation

- A **Student** may register for many **Courses**.
- A **Course** may have many **Students**.

Private, protected, and public symbols describe access inside each class.



If the registration relationship has attributes such as `registrationDate` or `grade`, introduce an **Enrollment** class

ERD vs Class Diagram

Criteria	ERD	Class Diagram
Focus on	Data to be stored and data relationships	Object classes, attributes, methods, and relationships
Core	Entity, attribute, key, relationship	Class, attribute, method, OO relationships
Applications for	Database design	Object-oriented software design
Behavior	Does not describe methods	Lists operations but does not describe detailed behavior flow
Question	What data does the system store?	What objects exist and how do they interact?

Mini Exercise:

Internship Registration System

Scenario :

A university wants to manage internship registration:

- A student may submit many internship requests over time, but each request is submitted to one company.
- Each request has a request date, start date, end date, and status.
- A lecturer reviews the request.
- A coordinator gives final approval.
- A request may include many uploaded documents.
- A company may host many students over time

Internship Registration System

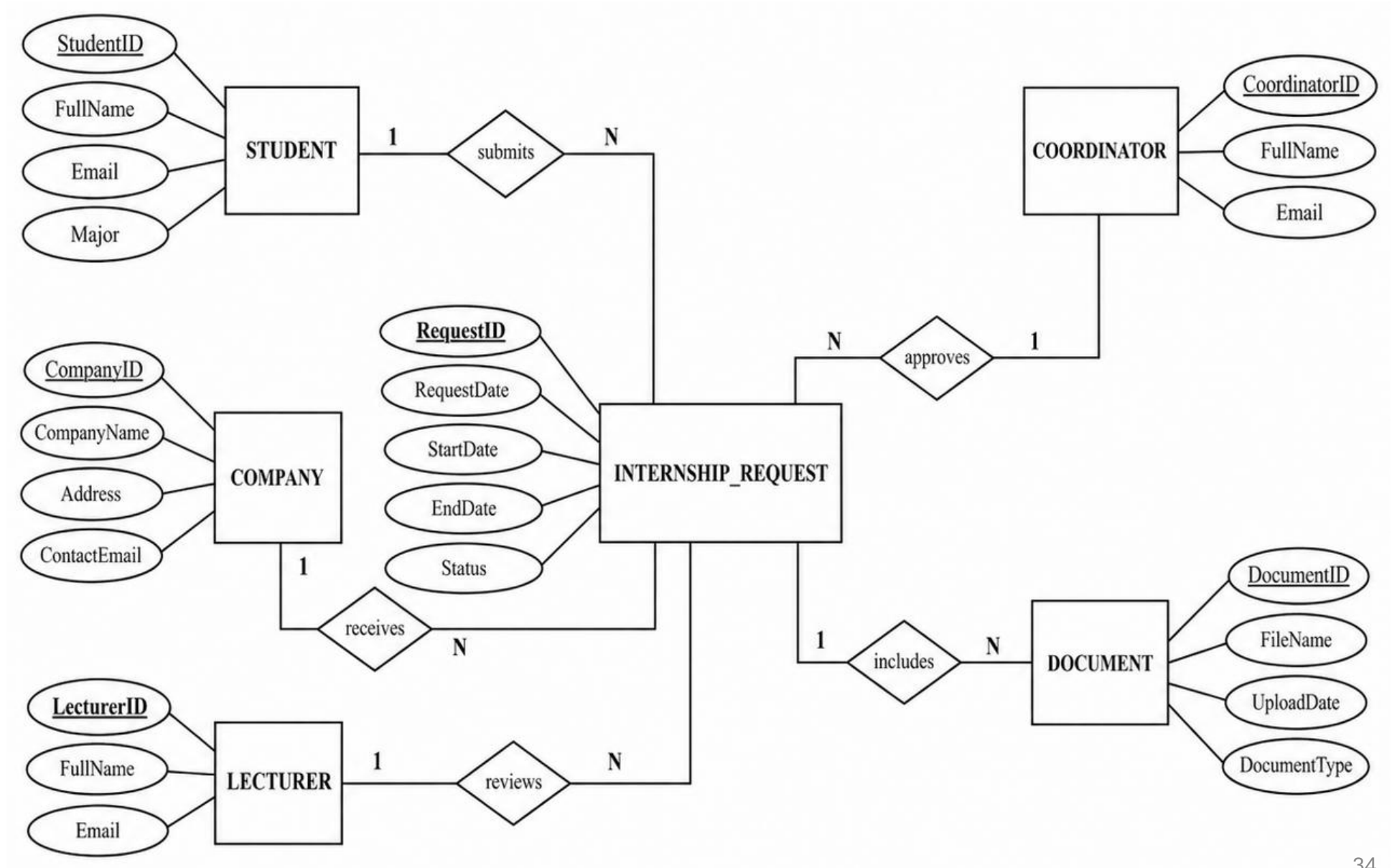
Tasks :

- Identify entities and keys
- Draw an ERD with cardinality
- Convert the main concepts into a simple Class Diagram
- Explain 2 differences between the two diagrams

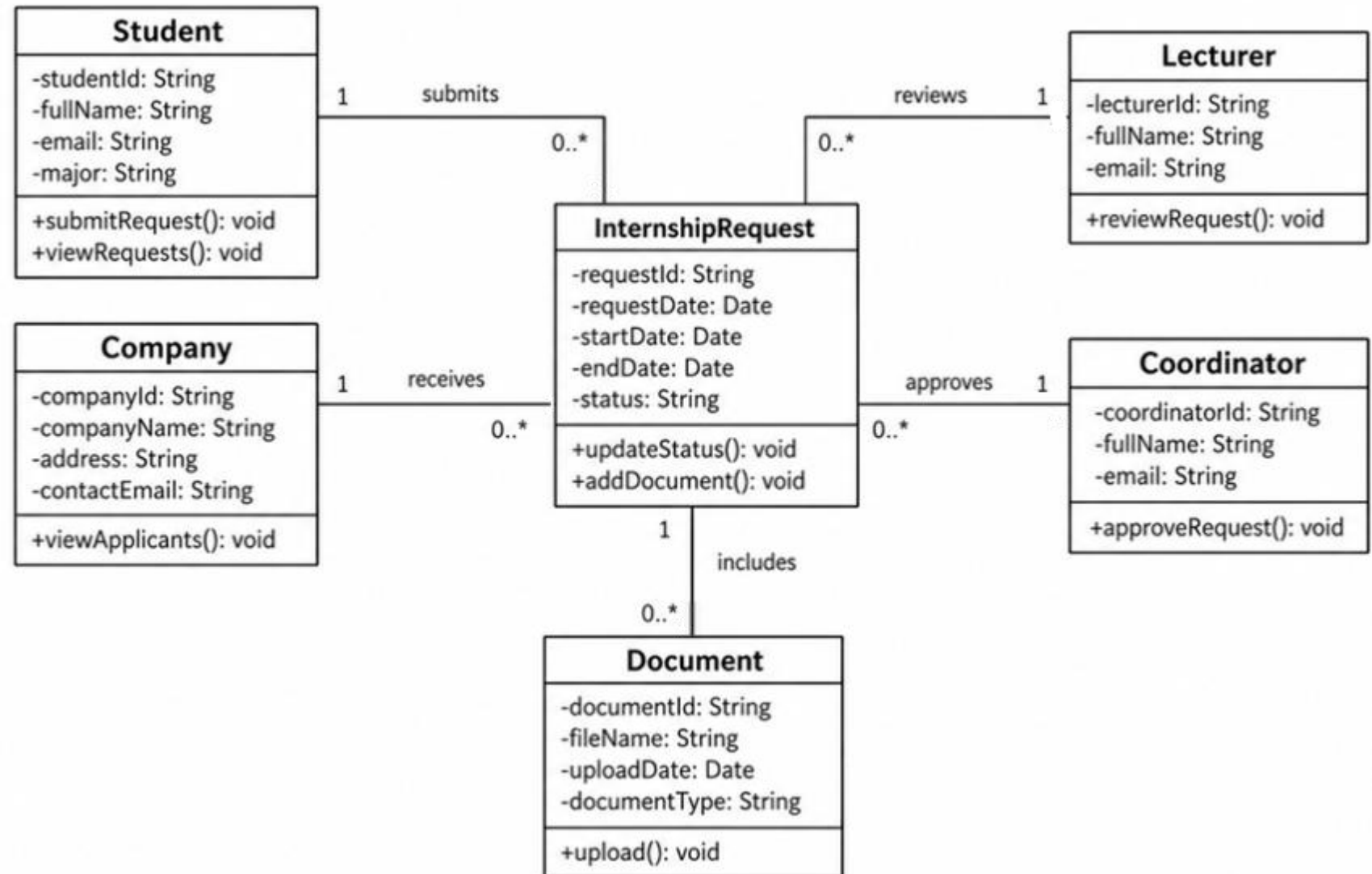
Expected output

- 1 ERD
- 1 basic Class Diagram
- Entities have keys, relationships have cardinality, class diagram has attributes/methods/multiplicity, explanation compares data vs object view

Internship Registration System (ERD)



Internship Registration System (Class Diagram)



SESSION 2

UML DIAGRAMS

UML Diagrams for System Analysis

This session introduces selected UML diagrams that help analysts describe system behavior and interactions.

Focus diagrams:

- Use Case Diagram: actor/use case/system boundary
- Activity Diagram: initial/final node/decision/swimlane
- Sequence Diagram: lifeline/message/activation
- State Diagram: state/transition/final state

**1. UML
overview**

2. Use Case

3. Activity

4. Sequence

5. State

Session 2 Learning Outcomes

By the end of this session, students should be able to:

- Explain the role of UML as a standard modeling language
- Identify the purpose and notation of selected UML diagrams
- Distinguish when to use use case, activity, sequence, and state diagrams
- Create a simple UML model from a short scenario

Remember

- Different UML diagrams answer different questions.

Unified Modeling Language (UML)

- UML is a **standardized graphical modeling language**.
- It is used to **visualize, specify, design, and document** software systems.
- UML provides a common visual language so teams can communicate consistently.
- Not every project needs every UML diagram.

Academic note

- UML is a language of notation, not a programming language.

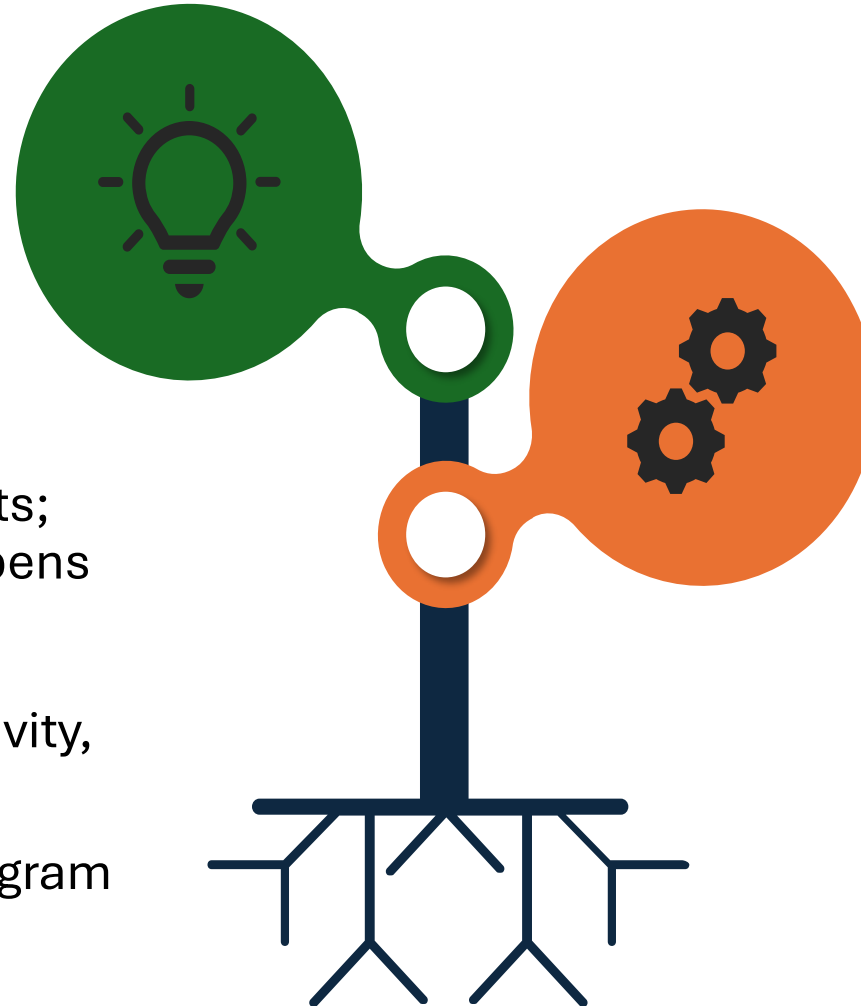
Structural vs Behavioral Diagrams

Structural diagrams

- Describe static structure.
- Example: Class Diagram

Key idea

- Structure tells us what exists;
Behavior tells us what happens
- Structural = Class Diagram
- Behavioral = Use Case, Activity, State
- Interaction = Sequence Diagram



Behavioral diagrams

- Describe actions, workflow, interaction, or change over time.
- Examples: Use Case, Activity, Sequence, State

Use Case Diagram

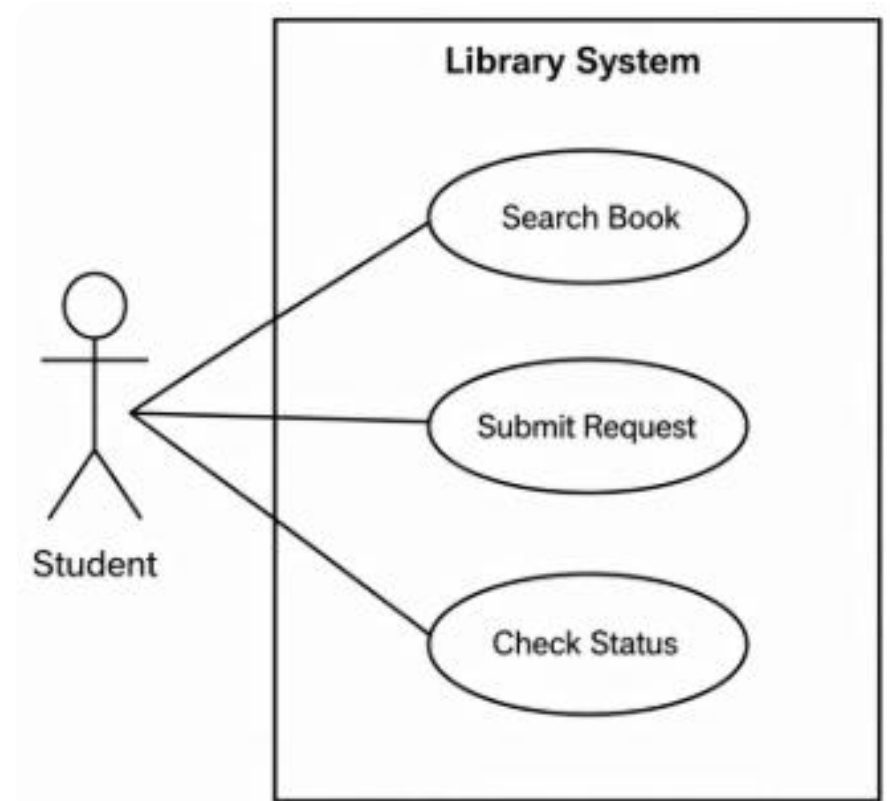
- A Use Case Diagram gives a **high-level view** of system functionality from the perspective of external actors.
- It shows **what the system does**, not how the internal logic is implemented.
- It is useful for defining **scope**, major goals, and actor-system interactions.
- It should be simple, readable, and stakeholder-friendly.
- It identifies external actors, major use cases, and the system boundary.

Main Elements of a Use Case Diagram

- **Actor:** a role outside the system
- **Use Case:** a goal-oriented function that provides value to an actor
- **Association:** communication between actor and use case
- **System Boundary:** defines what is inside the system and what is outside
- Relationships such as <<include>>, <<extend>>, and generalization; include = required reusable behavior; extend = optional/conditional behavior; generalization = specialized actor/use case

Remember

- Actors are outside the system.
- Use cases are inside the system boundary.
- Use Case Diagram = **what the system does for users**



Use Case

Scenario – Online Library Book Borrowing System

Current problem: At present, book borrowing at the university library is partly manual. Students often need to search for books, confirm whether a book is available, and ask librarians about the status of their requests. Librarians also need to manage requests and update records manually.

Target system behavior: To improve this process, the university plans to develop an Online Library Book Borrowing System. In this system, a student can log in, search for a book, view book details, and submit a borrowing request. The system checks whether the selected book is available. If the book is available, the system creates a borrowing record, sets its status to Requested, and sends the request to the librarian. If the book is not available, the system informs the student. A librarian can review borrowing requests and update the borrowing status to Approved, Borrowed, Returned, or Rejected. The student can log in at any time to check the borrowing status.

Main actors

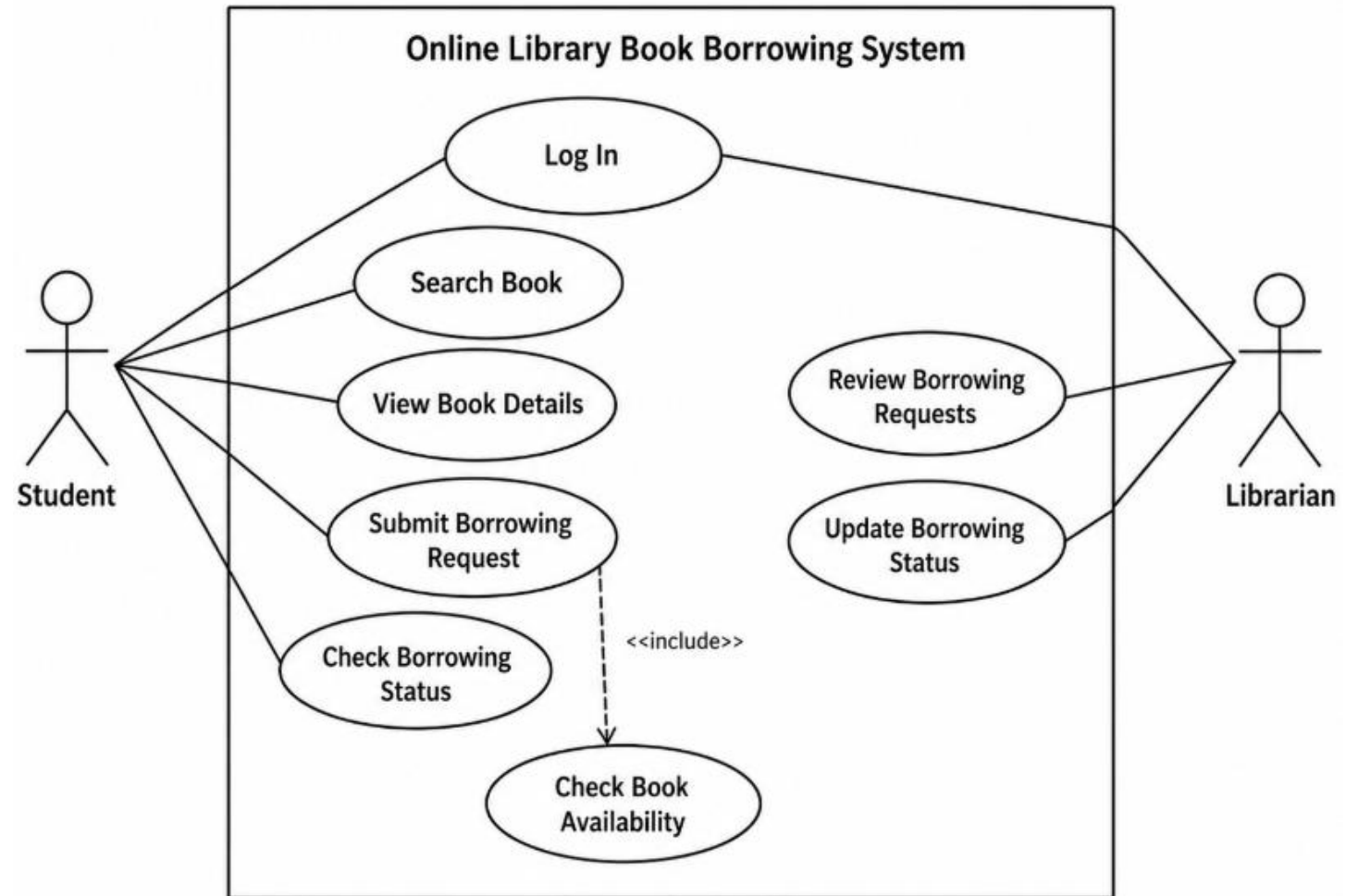
- Student
- Librarian

Use Case Diagram

How to draw

Step 1. Identify the actors • Student • Librarian	Step 3. Identify the system boundary • Online Library Book Borrowing System Step 4. Identify important relationships • Submit Borrowing Request <<include>> Check Book Availability
Step 2. Identify the main use cases • Log In • Search Book • View Book Details • Submit Borrowing Request • Check Borrowing Status • Review Borrowing Requests • Update Borrowing Status • Check Book Availability	Step 5. Draw the diagram • Draw actors outside the system boundary • Draw use cases inside the system boundary • Connect actors to related use cases using associations • Add <<include>> relationship where appropriate

Use Case Diagram - Online Library Book Borrowing System



What Is an Activity Diagram?

- An Activity Diagram models **workflow** and **step-by-step logic**; an Activity Diagram models the workflow of activities in a process from start to finish.
- It is used to describe:
 - The sequence of actions
 - Decision points
 - Alternative flows
 - Responsibilities of different participants
- Activity diagrams are useful for modeling business processes and system procedures.
- It is strong when analysts need to show **how a process proceeds**.

Activity Diagram

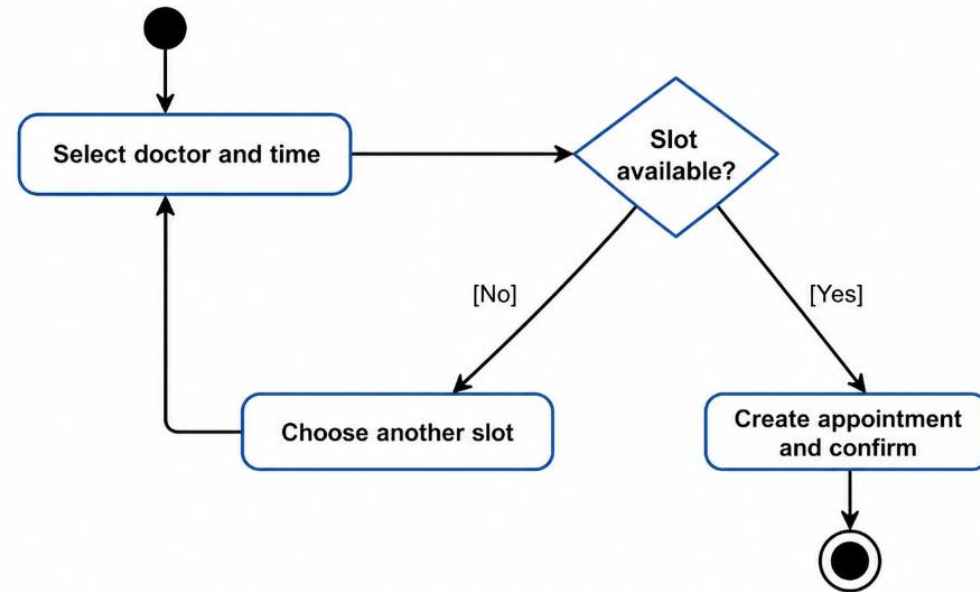
Main elements

- Initial node
- Action / activity
- Decision node
- Control flow
- Final node
- Swimlanes

Academic note

- Activity diagrams are especially useful when a use case is too general and needs detailed workflow analysis

Activity Diagram – Book Appointment Workflow



Activity Diagram

Scenario: Process a Transcript Request

In the Academic Service Request System, a student wants to request an academic transcript.

First, the student logs in to the system, selects the transcript request service, and fills in the request form. The student uploads the required documents and submits the request.

After that, the system records the request and sends it to the staff for review. The staff checks whether the request is complete.

If the request is incomplete, the staff marks it as **Needs Revision**, and the system notifies the student.

If the request is complete, the staff verifies the student record and processes the request.

When the transcript is ready, the system updates the request status to **Processed**, and the student is notified.

Activity Diagram

How to Draw the Activity Diagram

Step 1. Identify the process Process a Transcript Request Step 2. Identify participants / swimlanes • Student • Staff • System	Step 4. Identify decision points Is the request complete? • Yes → continue processing • No → notify student → student revises request → resubmit → staff reviews again.
Step 3. Identify major actions • Log in • Select request service • Fill in request form • Upload documents • Submit request • Record request • Review request • Verify student record • Update request status • Notify student	Step 5. Connect the flow from start to end • Start • Student submits request • Staff reviews request • System updates status • Student receives notification • End

Activity Diagram

Process a Transcript Request

Swimlanes

- Student
- Staff
- System

Main actions

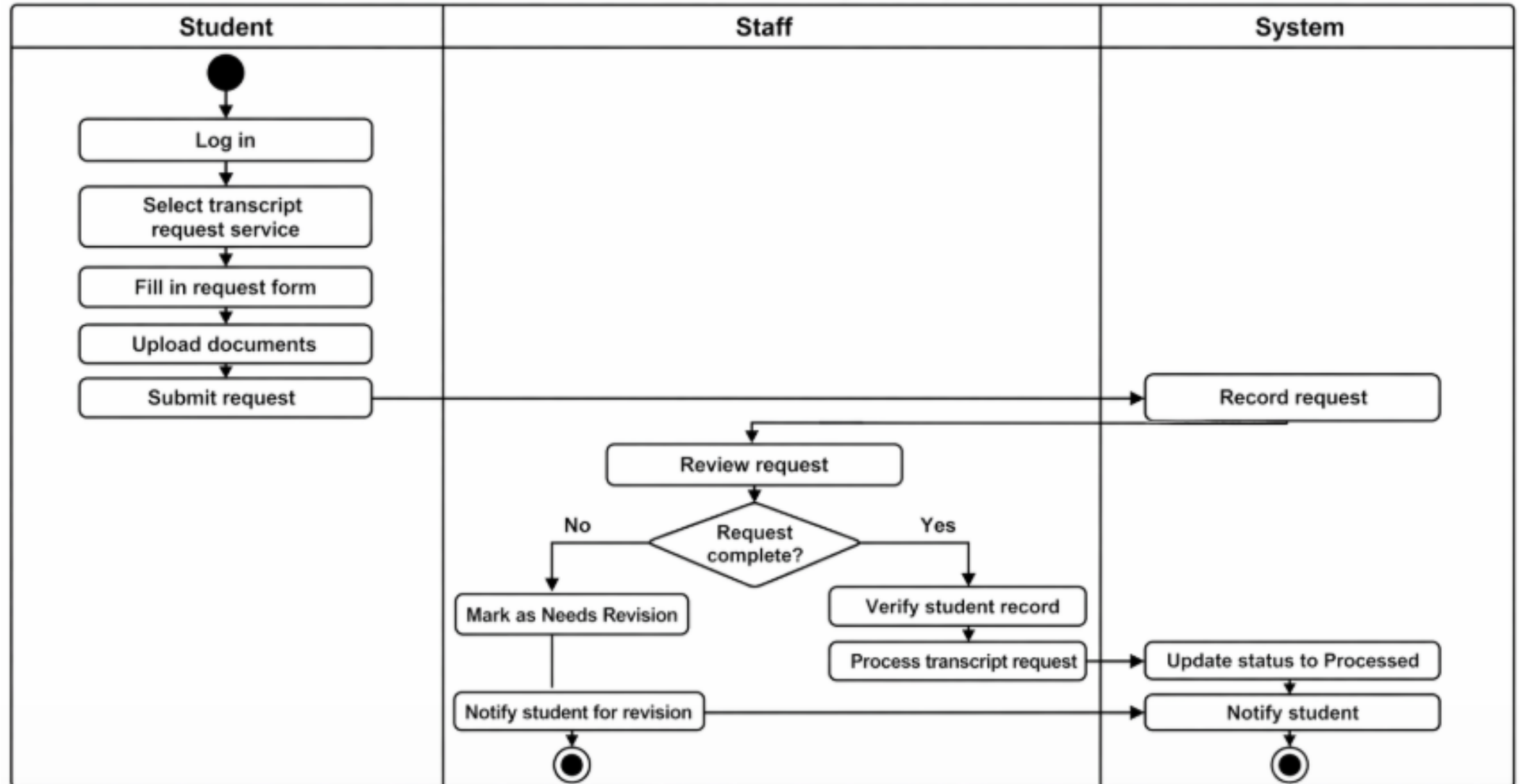
1. Log in
2. Select transcript request service
3. Fill in request form
4. Upload documents
5. Submit request
6. Record request
7. Review request
8. Check completeness
9. Verify student record
10. Update status
11. Notify student

Decision

- Is the request complete?

Activity Diagram

Process a Transcript Request



What Is a Sequence Diagram?

Sequence diagrams emphasize the message order between actors and objects along a timeline from top to bottom - **interaction over time**.

It focuses on:

- **Who** sends a message
- **To whom** the message is sent
- **In what order** the interaction happens
- A sequence diagram is useful when we want to describe the detailed interaction flow of **one specific use case**.
- It answers not only what happens, but **who talks to whom first, next, and after that** (Who calls whom? When do they call? How do they respond?)

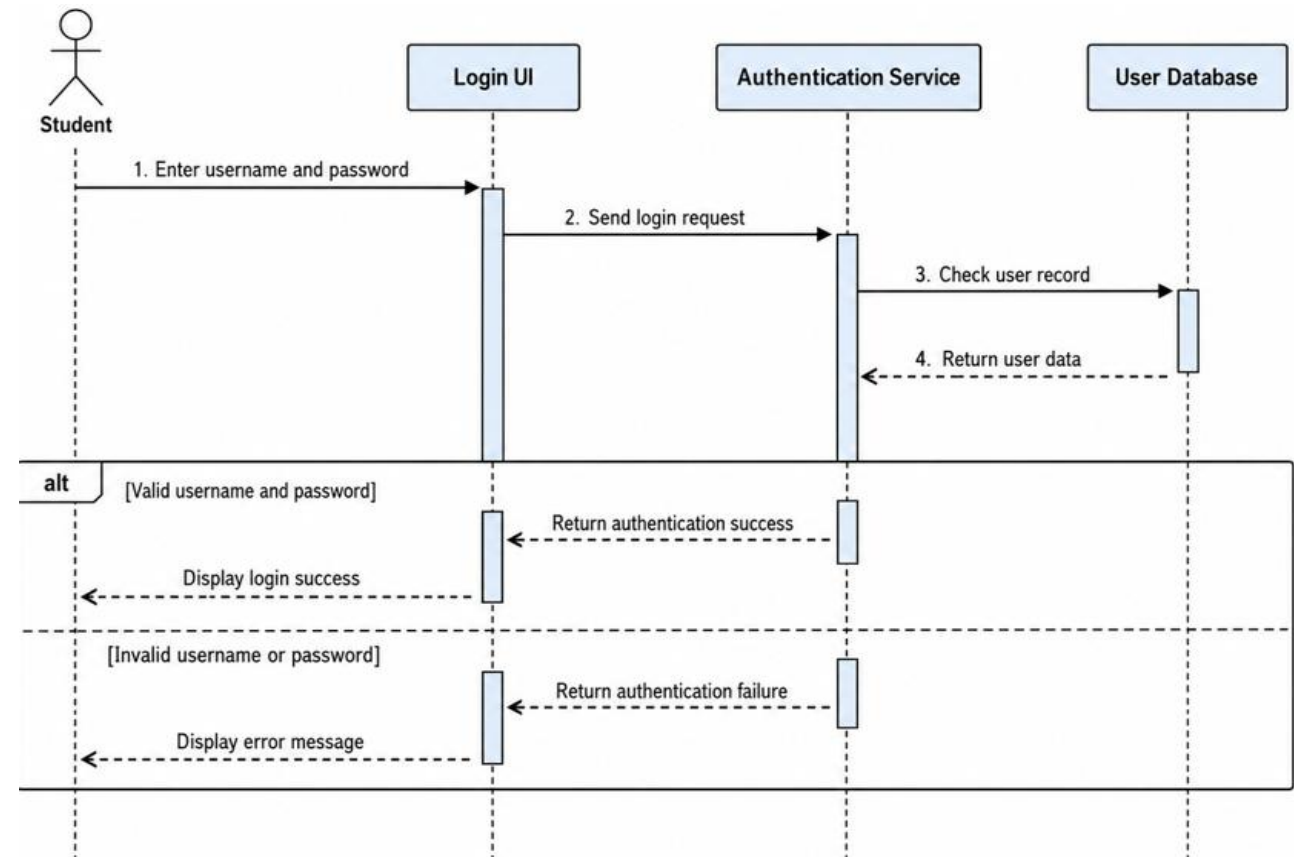
Main Elements of a Sequence Diagram

- **Actor / object**
- **Lifeline:** participant over time
- **Activation:** period object is executing
- **Message:** communication/call
- **Return message**
- Optional fragments such as **alt** (alternative branches), **opt**, **loop**

Academic note

- Sequence diagrams are strong for detailing one specific use case scenario.
- Suitable for describing which actor/object sends which message and in what order.

Sequence Diagram – Student Login



Sequence Diagram

Scenario: Student Checks Request Status

In the Academic Service Request System, a student wants to check the status of a previously submitted request. The student logs in and selects the Check Request Status function. The UI forwards the request to the status service. The service queries the database for the request record. The database returns the current status, and the system displays the result to the student.

Question this scenario answers

How does the system respond when a student checks request status?

Sequence Diagram

How to Draw the Sequence Diagram

Step 1. Identify the use case

Check Request Status

Step 2. Identify participants

- **Student** (actor)
- **UI** (boundary / interface)
- **Status Service** (control)
- **Database** (entity / data store)

Step 3. Arrange lifelines from left to right

Student → UI → Status Service → Database

Step 4. Identify messages in time order

1. Student sends status request
2. UI forwards request to Status Service
3. Status Service queries database
4. Database returns request record
5. Status Service sends status result to UI
6. UI displays current status to student

Step 5. Add optional alternative flow

If no request record is found, the system displays an error or “No request found” message.

Sequence Diagram

Student Checks Request Status

Participants

- Student
- UI
- Status Service
- Database

Main success flow

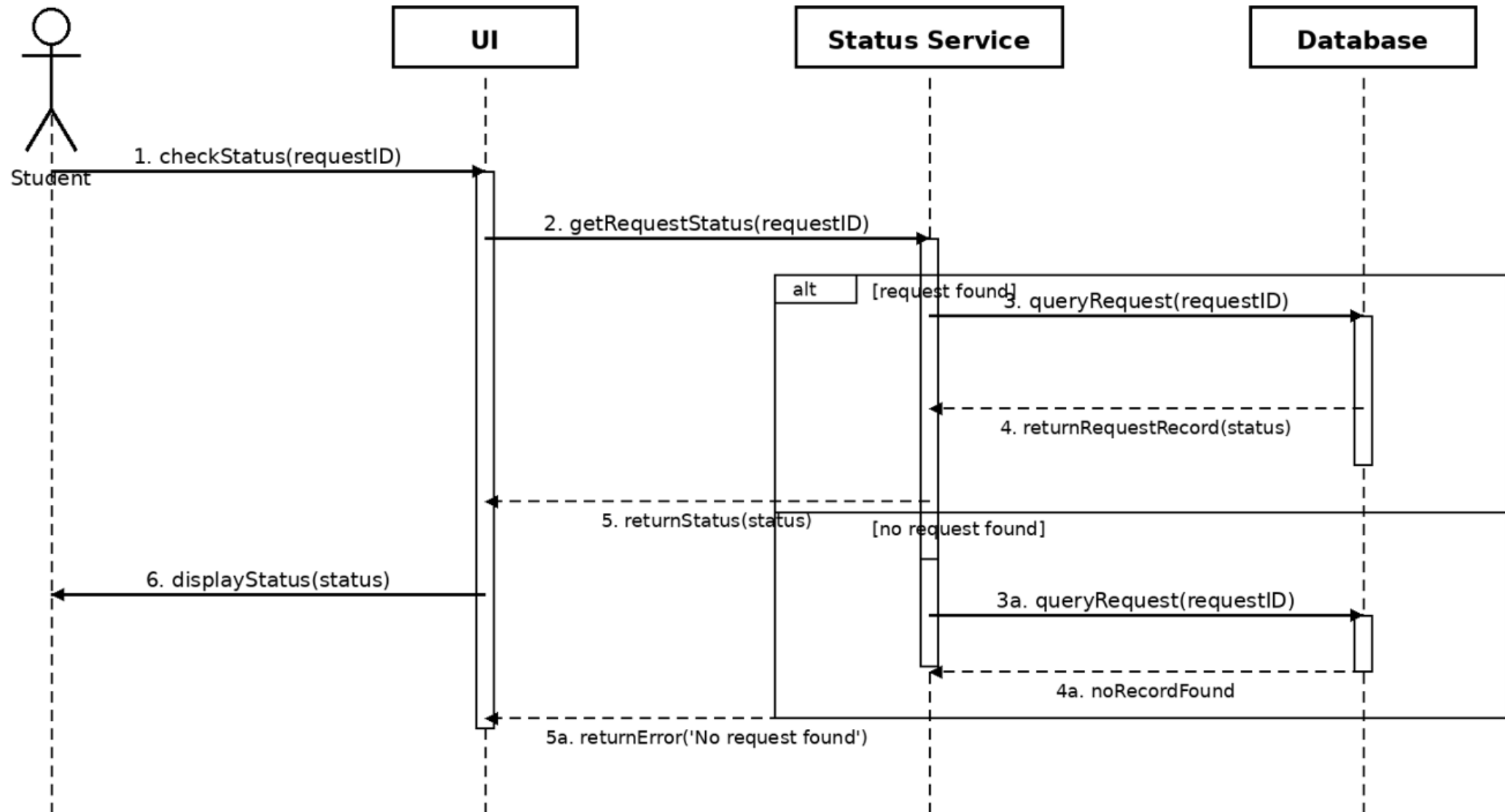
1. Student → UI:
 checkStatus(requestID)
2. UI → Status Service:
 getRequestStatus(requestID)
3. Status Service → Database:
 queryRequest(requestID)
4. Database → Status Service:
 returnRequestRecord(status)
5. Status Service → UI:
 returnStatus(status)
6. UI → Student: displayStatus(status)

Alternative flow

If requestID is not found:

- Database → Status Service: no record found
- Status Service → UI: return error message
- UI → Student: display “No request found”

Sequence Diagram - Student Checks Request Status



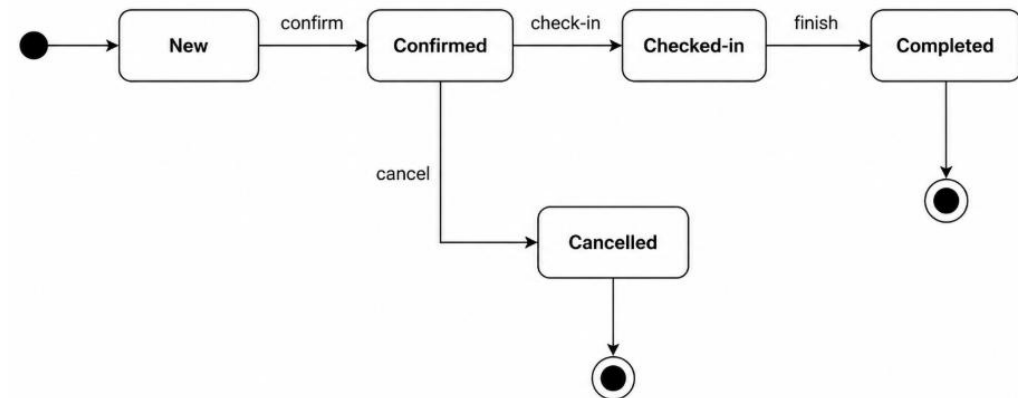
What is a State Diagram?

- A State Diagram describes an object transitioning from one state to another due to an event or condition
- It is useful for objects with a clear lifecycle and helps distinguish object states from process steps.
- Each state represents a meaningful condition of that object at a certain time.
- A transition happens when an **event** or **trigger** causes the object to move to a new state.
- State diagrams are useful for objects with a clear lifecycle such as requests, orders, applications, or tickets.

Remember box

- **Activity Diagram** = workflow of actions
- **State Diagram** = lifecycle of one object

State Diagram – Booking Status



State Diagram

Internship Request System

Scenario:

At a university, students must submit an internship request before starting their internship. First, a student creates a request and enters internship information such as company name, internship period, and required documents. At this stage, the request is still a **Draft**.

When the student clicks **Submit**, the request is sent to the university and becomes **Submitted**. After that, a lecturer or coordinator checks the information and the request moves to **Under Review**.

If the information is incomplete or unclear, the reviewer asks the student to revise the request. The request then changes to **Needs Revision**. After the student updates the information and submits it again, the request returns to **Submitted** and is reviewed again.

If the request satisfies all requirements, the coordinator approves it, and the request becomes **Approved**. After the internship is completed and all required documents are finalized, the request moves to **Completed**.

If the request does not meet requirements, it becomes **Rejected**.

State Diagram - Step to build diagram

Internship Request System

Step 1 – Define object

- Internship Request

Step 2 – Identify meaningful states

- Draft
- Submitted
- Under Review
- Needs Revision
- Approved
- Rejected
- Completed

Step 3 – Find the event that changes the state

- Submit
- Start review
- Request revision
- Resubmit
- Approve
- Reject
- Internship ends

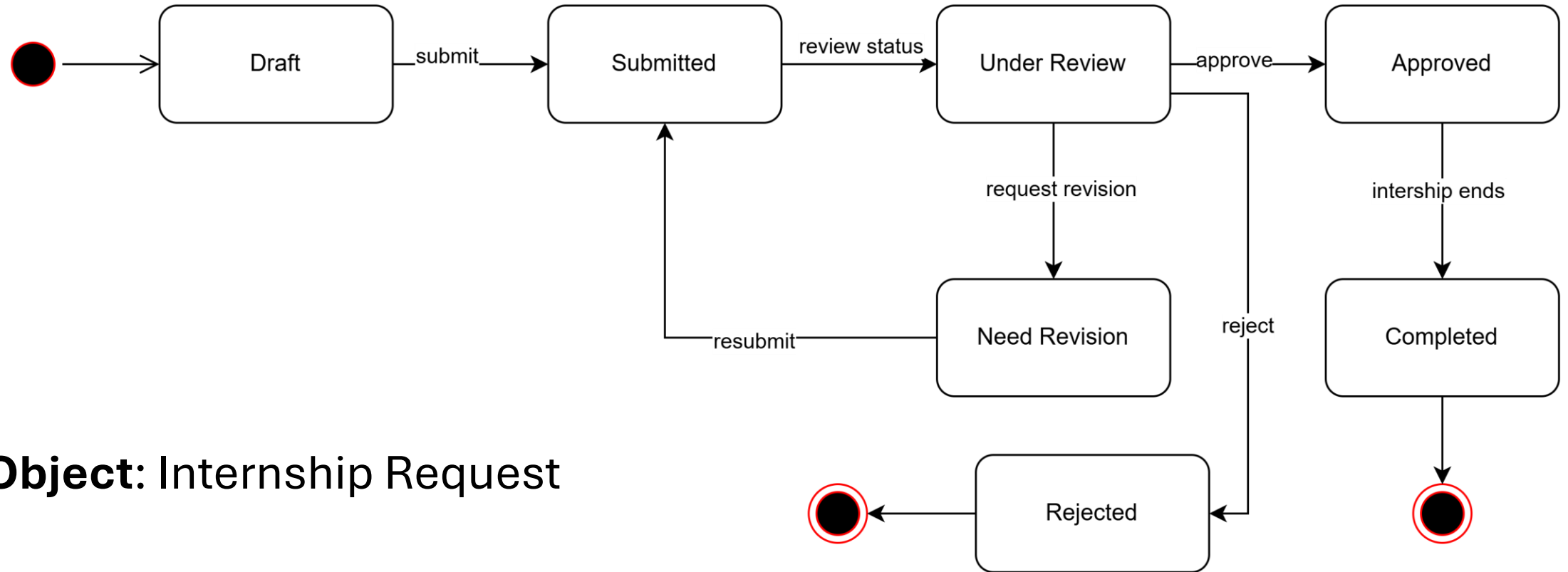
Step 4 – Identify the starting and ending points.

- Start → Draft
- Final states:
 - Rejected
 - Completed

Step 5 – Connecting states using transitions

- Use the arrow and write the trigger on the arrow.

State Diagram Internship Request



Object: Internship Request

Which UML Diagram Should We Use?

Modeling Question	Suitable Diagram
What data must the system store?	ERD
What classes and object relationships exist?	Class Diagram
Who uses the system and what goals do they have?	Use Case Diagram
What are the process steps from start to end?	Activity Diagram
Who sends messages to whom, and in what order?	Sequence Diagram
What states can one object move through?	State Diagram

Output Discussion

A strong answer should include:

- Correct actors and system boundary
- Meaningful use case names using verb phrases
- Proper use of <<include>> and <<extend>> where needed
- Clear decision points in activity flow
- Consistent labels and readable layout
- Sequence = one use case scenario, ordered messages, lifelines, alt frame.
- State = one object, meaningful states, event-triggered transitions

Common mistakes to discuss

- Actor drawn inside system
- Use case named as a noun
- Mixing process details into use case diagram
- Overcomplicated arrows

Conclusion

By the end of Week 04, students should understand that system analysis requires different modeling views.

- **ERD focuses on the data view of a system.**
It helps analysts identify entities, attributes, keys, relationships, cardinality, and optionality.
- **Class Diagram focuses on the object-oriented structural view.**
It shows classes, attributes, operations, visibility, multiplicity, and relationships among software objects.
- **UML diagrams support different analysis questions.**
Use Case Diagram describes system scope and user goals.
Activity Diagram describes workflow and decision logic.
Sequence Diagram describes message order between actors and objects.
State Diagram describes the lifecycle of one object.
- **A good analyst chooses the diagram based on the question being asked.**
More diagrams do not always mean better analysis; the right diagram must clarify the right problem.

Final Message

A diagram is not only a drawing. It is a reasoning tool that helps analysts understand, communicate, validate, and improve system requirements.